

Cross-Plane Operational Provenance for Cloud-Native Systems: Design and Reproducible Evaluation of a Lightweight Evidence-Aware Control Plane

Obede Bessa Rocha da Silva

Independent Cloud Solutions Architect and DevOps Researcher • Clermont, Florida, USA

Correspondence: obedebessa@gmail.com

ABSTRACT

Cloud-native operations distribute audit-relevant events across delivery, identity, orchestration, policy, telemetry, incident, and recovery systems. This paper presents an evidence-aware control plane (EACP), an asynchronous pattern that normalizes source metadata into an append-only service-level tuple while leaving full artifacts in their native systems and remaining outside the application request path by default. A Python and SQLite implementation was first evaluated against six separately indexed source tables using deterministic workloads of 10,000, 50,000, and 100,000 events, ten sequential seeded trials per size, and 600 reconstruction queries per trial. At 100,000 events, median amortized bulk EACP processing was 5.332 microseconds per event (187,550 events/s). Median p95 service-state query time was 0.691 ms versus 1.141 ms for the fragmented baseline (1.65×); correlation reconstruction was 0.022 ms versus 0.045 ms (2.05×). Every sampled result and the complete normalized projection matched, while the evidence database alone was approximately 181% of fragmented-database size. A second laboratory evaluation executed a single-node kind cluster with Kubernetes v1.36.1. Its sanitized namespace corpus contained 374 API-server audit events, including 132 events with an explicit operational correlation identifier and three audited RBAC denials with HTTP 403; EACP persisted all 374 unique auditID:stage keys and its append-only triggers rejected update and delete operations. The same frozen bytes were then replayed ten times each through EACP and OpenTelemetry Collector Contrib 0.158.0. Both paths retained 374 of 374 events in every run. An external post-export EACP validator normalized the retained bodies and matched 4,862/4,862 compared values to the reference projection; the Collector preserved raw audit bodies but did not natively generate the EACP projection. Median observed time to validated output was 60.659 ms for the tested EACP process and 622.477 ms for the tested Collector container, but asymmetric startup, host-observation, validation, and output semantics preclude a superiority claim. The study establishes local feasibility, executable Kubernetes integration, and explicit costs; it does not establish production effectiveness, field validity, or distributed-system scalability.

Keywords—*cloud-native systems; operational provenance; control plane; auditability; DevOps; evidence index; reproducibility*

1 Introduction

Cloud-native infrastructure is governed by several control planes at once. A deployment controller reconciles desired and observed state; an identity system decides whether an actor may change a resource; a policy engine permits or rejects a workload; a telemetry pipeline reports health; and incident and recovery systems record remedial action. Kubernetes explicitly models controllers as loops that move current state toward desired state [1], and its audit facility produces chronological, security-relevant records about cluster activity [2]. Yet an operator explaining why a service reached a particular state may still need to traverse multiple tools, identifiers, schemas, retention policies, and clocks.

This fragmentation matters for both operations and assurance. NIST SP 800-53 includes event logging, audit-record generation and protection, configuration-change control, continuous monitoring, incident handling, and contingency-planning controls [3]. The Cybersecurity Framework 2.0 organizes outcomes across Govern, Identify, Protect, Detect, Respond, and Recover [4]. These sources establish the need for accountable operational evidence, but they do not prescribe a small runtime index that binds a deployment, identity decision, policy result, alert, incident action, and recovery artifact at service granularity.

Existing specifications and systems solve adjacent problems. CloudEvents defines a vendor-neutral envelope and contextual attributes for event data [5]. OpenTelemetry defines a common log data model and correlation fields [6]. W3C PROV supplies a general model for entities, activities, and agents [7], [8]. Distributed tracing systems reconstruct request causality and performance [9], [10], while provenance-aware storage and whole-system provenance capture lower-level data lineage [11], [12]. EACP does not replace those mechanisms. It proposes a deliberately narrow operational layer that indexes the minimum metadata needed to reconstruct cross-plane service transitions and points back to authoritative source artifacts.

The central research question is: to what extent can a lightweight, asynchronous evidence-aware control plane correlate heterogeneous operational events and improve local state reconstruction while imposing explicit, bounded processing and storage costs? Four subquestions guide the pilot: (RQ1) which minimal fields are sufficient to reproduce known cross-plane event chains when identifiers are complete; (RQ2) what processing, storage, and indexed-query costs arise in a reference implementation; (RQ3) which trade-offs and failure conditions constrain interpretation; and (RQ4) can the mapping be exercised against records emitted by a live Kubernetes API server and preserve the same common projection as an OpenTelemetry Collector reference pipeline?

The paper contributes:

- **A service-level evidence model.** A normalized tuple binds source and observed time, actor, service, intent, policy, action, outcome, source pointer, correlation identifier, and tuple hash.

- **An off-path architecture.** Adapters, normalization, an append-only index, and a query API operate asynchronously by default and do not become a dependency of application requests.
- **A reproducible reference artifact.** Six heterogeneous indexed stores and the EACP index are implemented with Python and SQLite, with deterministic workload generation and row-equivalence assertions.
- **A transparent pilot evaluation.** Results report modest query gains together with a substantial physical storage cost, a single-node Kubernetes audit exercise, and a controlled comparison with an existing telemetry pipeline.

CLAIM BOUNDARY

The artifact demonstrates exact reconstruction for deterministic synthetic records with complete service and correlation identifiers and exercises one small, single-node Kubernetes audit corpus. It does not infer missing causality, establish tamper-proof source artifacts, or measure production-scale, network, concurrent-write, managed-control-plane, or application-request overhead.

2 Background and Related Work

2.1 Control planes, desired state, and audit records

Container orchestration illustrates why operational state is distributed. Kubernetes objects are records of intent, and controllers continually reconcile actual state toward that intent [1]. Its architecture reflects lessons from Borg and Omega about declarative configuration, reconciliation, and shared-state coordination [13]. Kubernetes auditing separately records who acted, what occurred, when it occurred, and which resource was affected [2]. Those records are valuable, but a production service can also be altered by external delivery systems, cloud identity providers, infrastructure APIs, policy engines, and recovery procedures. A cluster audit log therefore represents one evidence plane, not the entire operational history.

2.2 Event normalization and observability

CloudEvents addresses interoperability by standardizing event context such as id, source, type, time, and data schema [5]. OpenTelemetry addresses the representation and transport of logs, traces, and metrics; its stable log data model includes source and observed timestamps, resource attributes, event name, and optional trace context [6]. These standards inform EACP adapters and time semantics. EACP adds service intent, applicable policy, normalized action and outcome, a source-artifact pointer, and an operational correlation identifier because its target query is not merely ‘what was emitted?’ but ‘which decisions and observations explain this service transition?’ EACP may ingest CloudEvents or OpenTelemetry records without requiring that every source natively implements either.

The OpenTelemetry Collector is a vendor-neutral implementation for receiving, processing, and exporting telemetry [20]. Version 0.158.0 of the Contrib distribution provides a filelog receiver that can parse JSON records and a file exporter that serializes OTLP data [21], [22]. This study uses that Collector configuration as an existing reference pipeline for the shared tasks of reading, parsing, preserving, and exporting audit events. It is not treated as a functionally equivalent provenance database: the file exporter is not an indexed query store, and EACP-specific hashes and reconstruction indexes fall outside the common projection.

2.3 Provenance and distributed tracing

W3C PROV defines interoperable provenance in terms of entities, activities, agents, and their relationships [7], [8]. EACP can be projected onto that model: the actor is an agent, the operational action is an activity, and the source artifact is an entity. The EACP tuple is not a replacement serialization for PROV; it is a compact query index that can support a richer provenance export when necessary.

Dapper and Pivot Tracing demonstrate that correlation identifiers and dynamic causal queries can make distributed request behavior observable with controlled overhead [9], [10]. Their principal unit is a request or runtime causal path. EACP instead targets control-plane and operational transitions that may happen minutes or days apart and may have no shared trace span. PASS captures storage provenance automatically [11], and Linux Provenance Modules protect whole-system provenance [12]. Those approaches provide deeper lineage than EACP. The EACP trade-off is coarser service-level semantics, lower integration ambition, and retained pointers rather than replication of all source data.

2.4 Relationship to institutional governance research

The author's prior MGRDI article proposes an institutional governance and maturity model for cloud and DevOps adoption [19]. The present study addresses a different research object and method: it defines a systems artifact, implements executable data structures and invariants, and measures a technical benchmark. It does not reuse the MGRDI seven-layer structure, maturity scale, diagnostic lenses, executive indicators, or implementation roadmap. The separation is important: institutional governance can motivate the need for evidence, but it does not establish the performance or correctness of an evidence index.

3 Research Method and Design Requirements

The study follows a design-science orientation: identify an operational problem, derive objectives from authoritative and technical literature, construct an artifact, demonstrate it, and evaluate measurable properties [14], [15]. This version remains a laboratory technical pilot rather than a field evaluation. The artifact is judged against explicit requirements, a reproducible synthetic microbenchmark, a captured single-node Kubernetes audit workload, and a common-projection comparison with an OpenTelemetry

Collector reference pipeline; organizational usefulness, user effort, and production reliability remain future work.

Table 1. Design requirements and their operationalization

ID	Requirement	Operationalization in the reference artifact
R1	Heterogeneous input	Six deliberately different source schemas represent delivery, identity, policy, telemetry, incident, and recovery records.
R2	Source authority	The index stores a URI-like pointer; full logs and artifacts remain in their native systems.
R3	Service-level key	Every evidence row contains a normalized service identifier and source timestamp.
R4	Cross-plane correlation	An explicit correlation identifier links the six events in each synthetic operational chain.
R5	Idempotent append-only index	A unique (source_type, source_id) constraint prevents duplicate inserts; triggers reject UPDATE and DELETE.
R6	Off-path operation	Evidence processing is a separate database operation and is not invoked by an application request path.
R7	Detectable tuple alteration	SHA-256 covers a canonical representation of the stored tuple; it does not hash or authenticate the external artifact.
R8	Measurable cost	Processing time, physical bytes per event, and indexed p95 query latency are recorded at three workload sizes.
R9	Executable platform fit	A real kind cluster emits Kubernetes audit records, including denied RBAC operations, that are normalized and persisted.
R10	Existing-solution comparison	EACP and OpenTelemetry Collector 0.158.0 consume identical captured bytes and are checked against one neutral projection.

The requirements are intentionally testable but incomplete. R4 assumes that identifiers exist and are propagated correctly. R5 provides append-only behavior inside the SQLite application boundary, not cryptographic non-repudiation. R6 is an architectural placement decision, not a latency measurement. R7 detects a changed canonical tuple if a trusted digest is retained, but does not prove that the source

system or pointer target is trustworthy. R9 is a local laboratory exercise, and R10 is limited to the functions shared by both pipelines; neither requirement establishes field effectiveness or universal performance superiority.

4 EACP Architecture and Evidence Model

Figure 1 separates three concerns. Source planes retain authoritative records and expose native events. The evidence path maps those records to a common service-level index, correlates explicit identifiers, and exposes reconstruction queries. The application data path remains unchanged by default. Selective inline enforcement could be added for high-risk actions, but it is outside this artifact and would change the availability and latency analysis.

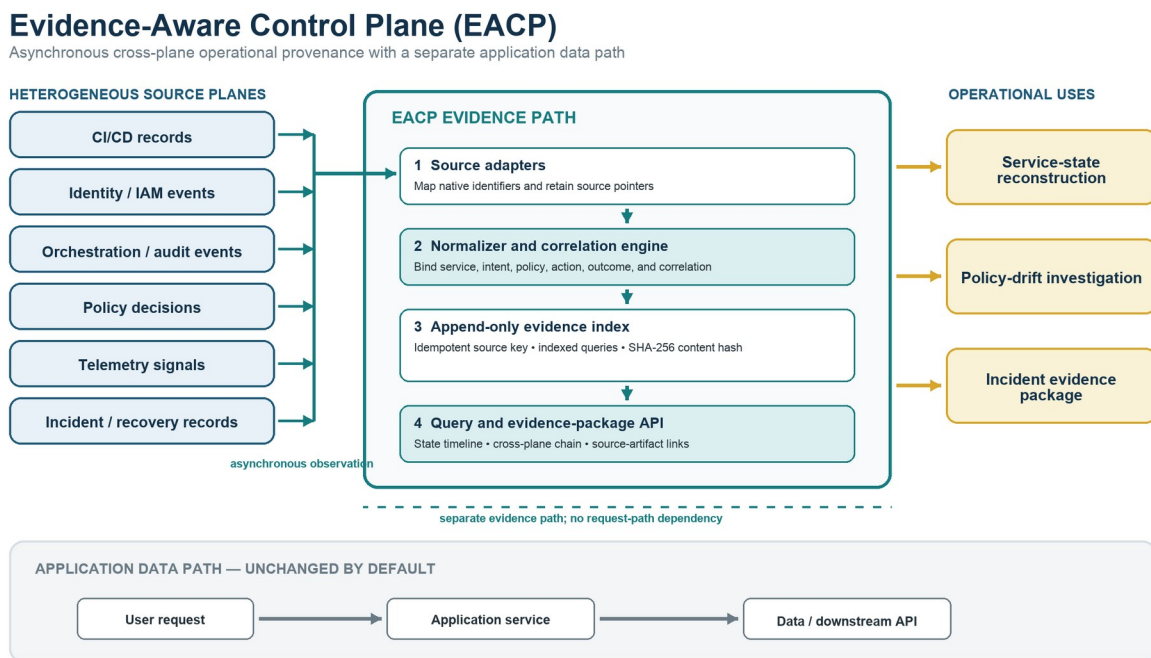


Figure 1. EACP observes heterogeneous source planes asynchronously, retains source pointers, and exposes operational reconstructions without becoming an application request-path dependency by default.

4.1 Components

- **Source adapters.** Adapters map native timestamps, actors, service identifiers, decisions, and artifact references. They should preserve source identifiers and avoid silently fabricating missing context.
- **Normalizer and correlation engine.** The normalizer emits the common tuple. This pilot uses explicit correlation identifiers; future adapters may use bounded inference, but inferred relationships must carry provenance and confidence.

- **Append-only evidence index.** The index supports idempotent insertion, service/time queries, correlation queries, and a digest of the canonical tuple. Retention and access control are deployment responsibilities.
- **Query and evidence-package API.** The API returns a time-ordered chain and the pointers required to retrieve authoritative artifacts. It should not imply that a pointer target is complete, available, or authentic unless the source provides those guarantees.

4.2 Tuple semantics

For an evidence-relevant event, EACP stores:

$$E = \langle t_s, t_o, a, s, i, p, x, o, u, c, h \rangle$$

where t_s is source time, t_o is observation time, a is actor, s is service, i is declared operational intent, p is the applicable policy or objective, x is the normalized action, o is outcome, u is a source pointer, c is an operational correlation identifier, and h is the SHA-256 digest of the canonical tuple. FIPS 180-4 defines SHA-256 as a secure hash algorithm for generating message digests [17]. In this pilot, h detects tuple changes only if the expected digest is protected elsewhere; it is not a digital signature and is not a digest of the source artifact.

The tuple is semantically compact because it indexes large artifacts instead of copying them. Physical compactness is a separate question: relational row headers, the digest string, uniqueness constraints, and three indexes can consume more bytes than the source records. This distinction is measured rather than assumed.

4.3 Invariants and query classes

The reference implementation enforces four invariants: (I1) a source event maps to at most one evidence row under the pair (source_type, source_id); (I2) normal update and delete statements are rejected; (I3) every row has a source pointer and correlation identifier; and (I4) service and correlation reconstructions are ordered by source time. The artifact evaluates two query classes. A service-state query reconstructs all indexed transitions for a service. A correlation query reconstructs one cross-plane chain. Policy-drift and incident-learning analyses can be built on those results but are not implemented here.

5 Threat, Privacy, and Failure Model

EACP assumes heterogeneous tools may omit context, clocks may differ, privileged operators may perform urgent manual changes, and source artifacts may expire. It does not assume that a database trigger defeats a privileged database administrator, that SHA-256 alone authenticates a record, or that a correlation identifier proves causality. The evidence index can itself reveal identities, services, incident

timing, recovery objectives, and sensitive source locations. Consequently, evidence collection creates a new security and privacy asset rather than eliminating governance work.

Table 2. Primary threats, mitigations, and residual limitations

Threat or failure	Design response	Residual limitation
Duplicate delivery	Unique source key and transactional insertion	Incorrect or unstable source identifiers can still collide or fragment one event.
Tuple alteration	Append-only triggers and canonical SHA-256 digest	A privileged administrator can bypass application controls; no signature or external ledger is provided.
Missing source artifact	Persistent pointer rather than copied payload	The pointer can expire, authorization can change, and the target may be deleted.
Clock skew	Separate source and observed timestamps	The pilot sets them equal and does not evaluate clock correction.
False correlation	Explicit correlation identifier in the pilot	Real deployments may have missing, reused, or incorrectly propagated identifiers.
Sensitive evidence	Minimum metadata, source-resident payloads, and separable access	Metadata can remain personal or security-sensitive and requires retention and access policies.
Evidence-path outage	Asynchronous, off-request-path placement	Events may be delayed or dropped unless adapters provide buffering, retries, and monitoring.

NIST cyber-resiliency guidance emphasizes the ability to anticipate, withstand, recover from, and adapt to adverse conditions [16]. EACP can contribute evidence for those activities, but it must itself be monitored, backed up, access-controlled, and tested. A production design should minimize personal data, segregate duties, define retention by event class, encrypt transport and storage, and record retrieval of sensitive evidence.

6 Reference Implementation

The reference artifact uses CPython's standard library and SQLite so that the entire pilot can run without cloud accounts or proprietary services. The fragmented baseline contains six tables with intentionally different field names. Each table is indexed on its native service/time pair and native correlation/time pair. This is stronger than a naive unindexed baseline and reduces the likelihood that EACP appears favorable merely because the alternative was poorly configured.

Table 3. Synthetic source planes and normalized semantics

Source plane	Native service field	Normalized intent	Example pointer scheme
Deployment	app	release_integrity	ci://runs/{id}

Source plane	Native service field	Normalized intent	Example pointer scheme
Identity	resource_service	least_privilege	iam://events/{id}
Policy	workload	policy_conformance	policy://decisions/{id}
Telemetry	target	service_availability	otel://queries/{id}
Incident	affected_service	incident_response	ticket://incidents/{id}
Recovery	system	recoverability	backup://checks/{id}

The evidence table stores source type and identifier, source and observed time, actor, service, intent, policy, action, outcome, pointer, correlation identifier, and a hexadecimal SHA-256 digest. It uses a uniqueness constraint for idempotency; indexes on (service, source_ts), (correlation_id, source_ts), and (source_type, source_ts); and triggers that abort UPDATE and DELETE. Inserts occur in one transaction using 1,000-row executemany batches. No payloads are copied, no network service is invoked, and no application traffic is generated.

The same query projection is implemented for both alternatives. The fragmented version performs a UNION ALL across six indexed tables and normalizes native fields in the SELECT clauses. The EACP version uses one indexed table. Both return the same ten fields in the same order. The script compares every measured query result row-for-row and raises an assertion if any value or ordering differs. It also streams the complete ordered projection from each alternative through SHA-256 and rejects the trial if the row count or digest differs. Representative SQLite EXPLAIN QUERY PLAN output is retained with the results to document index use.

A second adapter consumes Kubernetes audit.k8s.io/v1 JSON records. It maps auditID and stage to the source key; request and stage timestamps to source and observed time; the authenticated username to actor; object reference and workload labels to service; verb and HTTP response status to action and outcome; an audit URI to source pointer; and the explicit eacp.io/correlation-id annotation to operational correlation. The auditID identifies one API request and is not presented as a cross-plane chain identifier [23]. The public corpus is namespace-filtered and removes Kubernetes audit sourceIPs fields and local filesystem paths; the complete control-plane audit log remains local QA material.

For the existing-solution comparison, OpenTelemetry Collector Contrib 0.158.0 reads the same frozen JSONL bytes through the filelog receiver and exports OTLP JSON through the file exporter. The Collector retained each raw input line as the exported log body; the configured JSON parser also populated attributes, but those auxiliary attributes were not the input to the preservation check. After export, a shared external validator projects each recovered raw body to the same Kubernetes fields and compares count, field equality, and SHA-256 after stable ordering. The content hash is recomputed by that validator as a preservation check and is not attributed to the Collector; EACP's SQLite indexes have no Collector equivalent. Collector output size and EACP database size are reported descriptively but are not

interpreted as like-for-like storage efficiency because the former is an unindexed interchange file and the latter is an indexed evidence store.

7 Experimental Design

7.1 Workload

Three event volumes were evaluated: 10,000, 50,000, and 100,000. Every six consecutive events form a deterministic cross-plane chain—deployment, identity, policy, telemetry, incident, and recovery—with one service and correlation identifier. A coprime modular traversal balances consecutive chains across 200 services. For each trial seed, the smaller datasets are prefixes of the 100,000-event workload. Deterministic predicates introduce occasional failed deployments, access denials, degraded health, investigation states, and rollback outcomes. These variations exercise values and ordering; they are not probabilistic models of real incidents.

For each volume, ten sequential trials use recorded seeds. Each trial measures 300 randomly selected service reconstructions and 300 randomly selected correlation reconstructions, sampled with replacement. Both databases are warmed with representative indexed service queries before timing. Query order alternates between baseline-first and EACP-first to reduce systematic ordering bias. Elapsed time is measured with `perf_counter_ns`; database size is measured after `ANALYZE` and `VACUUM`.

7.2 Metrics and analysis

Evidence processing is an amortized bulk metric over one transaction and includes tuple normalization from pre-built Python dictionaries, canonical JSON creation, SHA-256 calculation, three evidence-index updates, and one transaction commit. It excludes raw-source insertion, source transport, collector buffering, and source-to-observer delay. Storage is the physical size of each separate SQLite database divided by event count. The EACP size is additional to the fragmented source database; it is not a replacement or savings. Query metrics are per-trial p50 and p95 values. The paper reports the median of ten trial-level values and shows interquartile ranges in Figure 2.

The benchmark is a deterministic microbenchmark rather than a sample from a defined production population. Accordingly, the analysis is descriptive. Formal hypothesis tests and confidence intervals would suggest a population model that the synthetic trials do not supply. Future production traces and randomized scenario experiments should preregister hypotheses and use paired inference appropriate to the deployment.

7.3 Execution environment

The reported run executed on an Apple M4 Max host with 16 logical CPUs and 51,539,607,552 bytes of memory, macOS 26.5.1 arm64, CPython 3.11.9, and SQLite 3.51.0. The benchmark is single-process.

Both databases use `journal_mode=DELETE`, `synchronous=FULL`, `temp_store=MEMORY`, foreign keys enabled, and a 64 MiB SQLite cache setting. The exact command, environment metadata, trial-level CSV, and summary JSON accompany the manuscript.

7.4 Kubernetes laboratory and reference-pipeline protocol

The platform exercise used kind 0.32.0 with a single Kubernetes v1.36.1 control-plane node. API-server auditing was enabled through a mounted audit policy [2], [24]. Secrets and authentication/authorization review APIs were excluded, RequestReceived was omitted, and request/response capture was restricted to synthetic non-secret resources in namespace `eacp-k8s-eval`. Three workload rounds created, labeled, read, listed, and deleted ConfigMaps, patched a Deployment, and issued one unauthorized patch per round through a constrained service account. The driver required each unauthorized request to fail and the audit analyzer required corresponding HTTP 403 records.

After capture, the namespace-filtered corpus was frozen before comparison. Ten fresh EACP SQLite ingestions and ten fresh OpenTelemetry Collector Contrib 0.158.0 replays [25] consumed identical bytes on the same host. Each replay started with empty output state; completion required the expected event count, a valid serialized output, and projection equality. Elapsed wall time includes pipeline startup and completion detection, so fixed container startup costs can dominate this compact corpus. Trial summaries report median, interquartile range, minimum, and maximum. These sequential same-host replays are descriptive repetitions rather than independent samples from a production population.

8 Results

Table 4. Median evidence-processing and physical-storage results across ten trials

Events	Ingest μ s / event	Events / second	Fragmented bytes / event	EACP bytes / event (EACP / fragmented)
10,000	4.955	201,839	233.9	414.9 (177.4%)
50,000	5.062	197,558	227.7	412.8 (181.3%)
100,000	5.332	187,550	228.4	414.0 (181.3%)

Amortized bulk processing remained near 5.0–5.3 microseconds per event across the three volumes, or approximately 187,550–201,839 events/s in the measured single-process configuration. This is a local bulk-processing result, not per-event latency or end-to-end ingestion throughput. The physical evidence database consumed approximately 413–415 bytes per event. Because the fragmented source database used about 228–234 bytes per event, the evidence store was 77–81% larger; retaining both produces a combined footprint approximately 2.77–2.81 times the source-only baseline.

Table 5. Median p95 reconstruction-query results across ten trials

Events	Query	Fragmented p95 (ms)	EACP p95 (ms)	Median speedup
10,000	Service state	0.105	0.075	1.38×
10,000	Correlation chain	0.028	0.019	1.53×
50,000	Service state	0.510	0.336	1.51×
50,000	Correlation chain	0.036	0.020	1.80×
100,000	Service state	1.141	0.691	1.65×
100,000	Correlation chain	0.045	0.022	2.05×

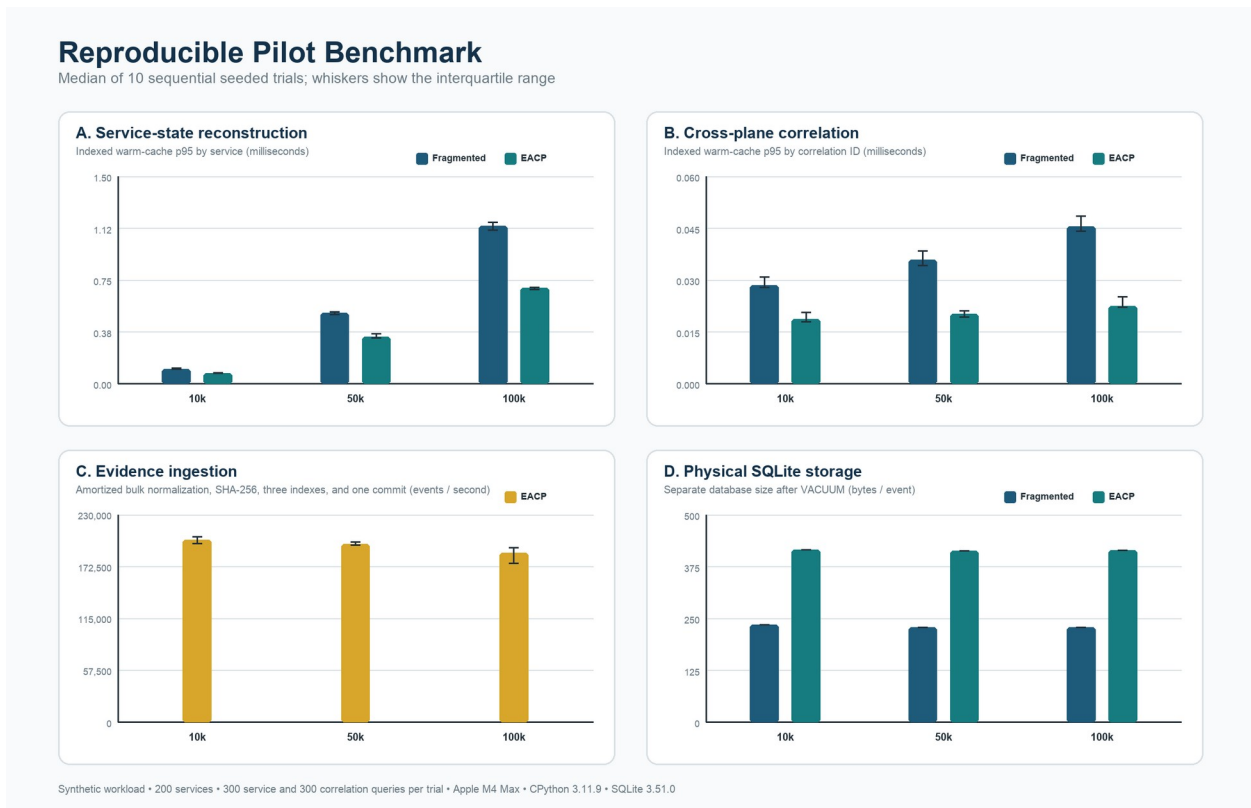


Figure 2. Pilot benchmark results. Bars show the median of ten trial-level metrics and whiskers show the interquartile range. The EACP database is an additional evidence index, not a replacement for source storage.

8.1 RQ1: reconstruction adequacy

For the generated workload, the tuple was sufficient to reproduce the measured service and correlation projections exactly. Every one of the 600 sampled query pairs in every trial was compared row-for-row, and all assertions passed. The complete ordered projection also matched in row count and SHA-256 digest for all 30 trials. This result demonstrates schema and query equivalence under complete explicit identifiers. It does not estimate precision or recall for inferred correlation, because the prototype performs no inference and the workload contains no missing or ambiguous identifiers.

8.2 RQ2: processing, storage, and query cost

The evidence path added measurable local processing and a substantial storage footprint. Query gains were consistent but modest: service-state p95 speedup increased from 1.38× at 10,000 events to 1.65× at 100,000, and correlation speedup from 1.53× to 2.05×. Average rows returned by a service query increased from approximately 50 to 250 and 500 across the three nested workloads; each correlation query returned six rows. Absolute timings remained below 1.2 ms because both alternatives were local, indexed, and warm-cache. A production environment with separate remote backends could show a larger coordination benefit, a smaller database benefit, or both; this benchmark cannot decide which.

8.3 RQ3: trade-off

The principal trade-off is not ‘more evidence for free.’ Normalization reduces query fan-out and gives one service-level contract, but retaining the source database and the additional EACP store raises combined physical storage to about 2.8 times the synthetic source-only database in this pilot. The digest and three indexes materially contribute to that cost. Implementers should select fields and indexes from actual query requirements, measure retention tiers, and consider storing binary digests rather than hexadecimal text. The pilot supports feasibility with explicit costs; it does not support a universal claim that EACP is cheaper than querying native tools.

9 Laboratory Kubernetes Evaluation Against an OpenTelemetry Collector Reference Pipeline

9.1 RQ4: captured Kubernetes audit evidence

Table 6. Single-node Kubernetes audit exercise and EACP checks

Measured item	Observed result	Interpretation boundary
Namespace audit corpus	374 events; 373 audit IDs; 374 auditID:stage keys	One request emitted both ResponseStarted and ResponseComplete.
Explicit operational correlation	132 rows tagged eacp-round-01/02/03	Only actions carrying the explicit annotation are counted.
RBAC policy denial	3 HTTP 403 events: 2 preliminary Deployment GETs and 1 ConfigMap PATCH	The workload issued three unauthorized patch commands, but these are the verbs audited as denied.
Persistence integrity	374 stored rows; input count matched; UPDATE and DELETE blocked	SQLite application-boundary controls, not cryptographic immutability.
Local EACP persistence	Median 3.002 ms over 10 trials (8.026 μs/event)	Small frozen corpus; excludes network and capture delay.

The live API server produced 1,819 total audit records; after excluding a service-account token subresource record, 374 belonged to the evaluation namespace and formed the public, sanitized corpus. Of those, 338 were initiated by kubectl, 132 retained an explicit workload-round correlation annotation, and three represented the expected authorization denials: two preliminary Deployment GETs and one ConfigMap PATCH. EACP stored every auditID:stage key, preserved 374 distinct hashes of sanitized canonical tuples, and returned indexed service and correlation reconstructions. This establishes executable compatibility with one Kubernetes audit configuration; it does not establish cross-plane completeness because the capture covers only the Kubernetes API plane.

9.2 Existing-solution reference comparison

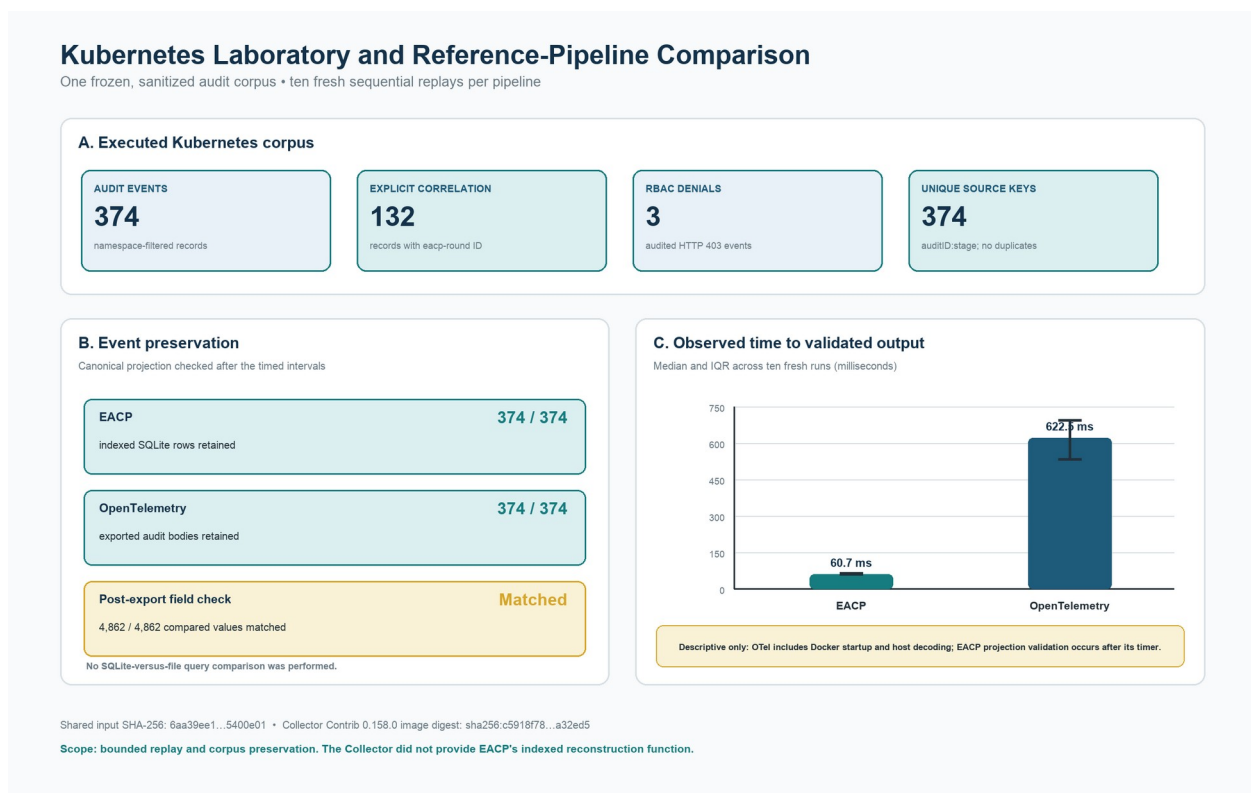


Figure 3. Kubernetes corpus, event-preservation checks, and descriptive observed time to validated output for ten fresh runs per pipeline. Timing and output-size differences are not interpreted as intrinsic superiority because runtime startup, host-side output observation, validation placement, and output semantics differ.

Table 7. Descriptive paired replay results over the same 374-event corpus

Pipeline	Median observed-output time [IQR]	Amortized completion rate	Events retained	Output form and median size
EACP	60.659 ms [59.940–63.602]	6,165.72 events/s	374 / 374	Indexed SQLite; 290,816 bytes
OpenTelemetry Collector	622.477 ms [532.604–692.480]	600.98 events/s	374 / 374	Unindexed OTLP/JSON; 3,338,502 bytes

All ten EACP runs and all ten Collector runs retained 374 of 374 events with no duplicate source keys. The Collector retained each raw line as the exported log body; its configured parser also populated attributes, but the post-export EACP validator operated on the retained body. That shared external validator—not the Collector configuration—mapped the recovered audit objects to the 13-field canonical projection. All 4,862/4,862 compared values matched the reference CSV generated separately by the Kubernetes analyzer, and every run produced projection digest 196d4a1bf8d057d9fe9e6f18062b7c5ac5228642df3098b28c84fb48d7a67da6. This is evidence of content preservation through the configured Collector path; it is not evidence that OpenTelemetry natively generated the EACP semantics.

Table 7 and Figure 3 report the measured configurations rather than a winner. The Collector boundary starts with Docker-container creation and ends after the host has observed and decoded the complete unindexed OTLP/JSON export; the EACP boundary includes a fresh Python process through its indexed SQLite commit, while database projection reading occurs later during validation. Their byte footprints and timing boundaries are therefore unlike, and no compression or speed ranking is inferred. Likewise, the study does not compare EACP indexed queries with file scans. Within the overlapping ingest-and-preserve scope, both paths retained the complete frozen corpus; only EACP's separate evidence-index function was measured for reconstruction queries.

10 Discussion

10.1 Design implications

First, cross-plane provenance can be treated as an index rather than a centralized log lake. Keeping pointers preserves source ownership and limits payload duplication, while a normalized service key reduces repeated schema translation. This approach depends on the continued availability and authority of source systems, so pointer health and retention compatibility become explicit operational controls.

Second, asynchronous placement is an availability choice. An evidence outage need not block application traffic, but it can create blind periods. Production adapters therefore require durable queues, retry and dead-letter behavior, source checkpoints, lag objectives, and completeness monitoring. Google SRE practice emphasizes explicit service objectives and error budgets [18]; an EACP deployment should define similar objectives for evidence delay and loss rather than assume that asynchronous means reliable.

Third, cryptographic language must remain precise. The current digest can reveal that a canonical tuple no longer matches an expected digest. It does not establish authorship, trusted time, external immutability, or artifact integrity. Stronger assurance could use keyed authentication, digital signatures, transparency logs, or write-once storage, but each adds key-management, cost, privacy, and recovery obligations.

10.2 Deployment path

A prudent deployment would begin with one service and two or three sources, define a stable service catalog, select evidence-relevant event classes, and verify pointer retention. The next step would add explicit correlation propagation through CI/CD, policy, and incident systems. Operators should first run the EACP in observation-only mode, compare reconstructed chains with native sources, and measure missingness, delay, false joins, and storage. Inline enforcement should be evaluated separately because it changes the threat and failure model.

The architecture aligns with audit and accountability, configuration management, continuous monitoring, incident response, and contingency-planning outcomes in NIST SP 800-53 [3], but alignment is not certification. An organization must still define controls, assess implementation evidence, meet applicable legal and contractual requirements, and decide whether the evidence index itself falls within regulated scope.

11 Threats to Validity

11.1 Construct validity

The benchmark measures database reconstruction time, not human incident-reconstruction effort, audit quality, compliance, resilience, or organizational value. The six synthetic schemas approximate source heterogeneity, while the platform exercise implements only the Kubernetes API audit plane; cloud, CI/CD, external IAM, telemetry, ticketing, and recovery APIs remain simulated or absent. Explicit correlation IDs make correctness easier than many real environments. The term ‘evidence’ means an indexed operational record and pointer; it does not mean legally admissible proof.

11.2 Internal validity

The fragmented and EACP microbenchmark alternatives use SQLite but different physical schemas. The baseline requires six UNION branches, while EACP uses one table; this is the intended design difference, yet query-planner behavior and cache state can affect timings. Warm-up, alternating execution order, identical pragmas, indexed baseline tables, ten seeds, and row-equivalence assertions reduce but do not eliminate bias. In the Collector comparison, Python/SQLite, a Go container, batching, serialization, and startup semantics differ; wall time therefore describes the tested end-to-end configurations rather than an intrinsic algorithmic property. Background processes were not controlled. No concurrency, failure injection, or cold-cache isolation was performed.

11.3 External and conclusion validity

Results from one Apple Silicon host, one single-node local kind cluster, SQLite, and one compact audit corpus cannot be generalized to distributed databases, managed Kubernetes, event buses, remote

vendor APIs, production retention, or high-cardinality services. The synthetic event distribution is not calibrated to empirical traces. Trial medians and interquartile ranges describe these runs only; they are not estimates of a production population. A real deployment could be dominated by network calls and operator coordination rather than local database or container execution.

11.4 Researcher and artifact limitations

The architecture, implementation, workload, and interpretation were produced within one project and were not independently replicated or peer reviewed. The workload generator and EACP adapter share assumptions about identifiers and field mappings; projection equality confirms implementation agreement for the frozen corpus, not semantic completeness. Independent reimplementations, adversarial test cases, multi-node and managed-cluster repetitions, and field data are required before stronger claims.

12 Reproducibility and Data Availability

The accompanying artifact contains the synthetic benchmark, Kubernetes manifests and audit analyzer, the OpenTelemetry Collector configuration and comparison runner, figure generator, trial-level CSV, summary JSON, environment metadata, representative query plans, privacy notes, checksums, README files, and citation metadata. Analysis scripts use Python's standard library; the figure generator requires Pillow. Kubernetes capture additionally requires Docker, kind, and kubectl. The reported synthetic command is:

```
python3 eacp_benchmark.py \
--sizes 10 000 50 000 100 000 \
--trials 10 \
--services 200 \
--query-samples 300 \
--output results
```

The Kubernetes and frozen-corpus comparison entry points are:

```
./experiments/kubernetes/run_experiment.sh
python3 experiments/comparison/opentelemetry/run_comparison.py run \
--input <public-audit.json> \
--reference-csv <normalized.csv> \
--output-dir <results> \
--trials 10
```

The random seeds, exact software versions, container digests, privacy transformations, trial outputs, and release checksums are recorded. The public repository for the version 1.1.0 artifact is <https://github.com/obedebessa/eacp-operational-provenance>. Its permanent archival identifier is the Zenodo software/artifact DOI <https://doi.org/10.5281/zenodo.21817377>. This DOI identifies the versioned software, configuration, data, documentation, figures, and reproducibility materials; it is not an article DOI for this preprint. The repository and archival record identify this manuscript as a preprint / technical report and do not imply peer review. A later journal or conference submission should cite the versioned artifact, preserve raw results, and disclose modifications to the workload or analysis.

13 Declarations

Ethics and data

The study uses deterministic synthetic operational records and a local ephemeral Kubernetes cluster populated only with synthetic resources. The public audit subset was filtered and sanitized to remove Kubernetes audit sourceIPs fields, credential identifiers, certificate material, token projections, and local paths. Synthetic internal cluster addresses remain where they are part of resource status. It includes no human participants, private organizational logs, or production credentials.

Funding and competing interests

No external funding was received for this work. The author declares no competing interests.

AI-use disclosure

OpenAI ChatGPT/Codex was used to assist with literature-discovery support, code scaffolding, benchmark-execution assistance, language drafting, document layout, and editorial organization. The named author remains responsible for independently reviewing the artifact, verifying claims and references, and accepting the final content before public dissemination. No AI-generated empirical observations were substituted for executed benchmark results; the reported values are drawn from the accompanying trial files.

14 Conclusion

EACP frames cloud-native governability as a cross-plane operational-provenance problem. The proposed pattern leaves authoritative artifacts in their source systems, normalizes a small service-level contract, and keeps evidence processing outside the application request path by default. In the reproducible synthetic pilot, the index matched the fragmented baseline for every sampled reconstruction and for the complete normalized projection, and produced modest p95 query speedups while amortized bulk processing measured about 188,000–202,000 events/s. The measured storage result is equally important: a heavily indexed, hexadecimal-hash evidence store used about 413–415 bytes per event, 77–81% more than the fragmented source database, with a combined footprint near 2.8 times source-only storage.

The Kubernetes exercise adds a practical but bounded result: the adapter normalized and stored all 374 sanitized API-server records, including explicit correlation metadata and three real RBAC denials. The OpenTelemetry reference pipeline retained the same 374 events in all ten replays, and an external post-export validator matched 4,862/4,862 compared values after normalizing the retained bodies. This confirms shared-corpus preservation, not Collector-native EACP normalization or platform superiority.

Observed times to validated output and byte footprints describe unlike tested configurations and are not generalized.

The defensible conclusion is therefore limited but useful. The study demonstrates offline normalization of a captured audit corpus emitted by a running, single-node local kind cluster and content preservation through one version-fixed OpenTelemetry reference pipeline. A separate evidence index can simplify local reconstruction queries, but it has storage and assurance costs and does not solve missing identifiers, source retention, privacy, or distributed-system reliability; these results do not establish streaming ingestion, production effectiveness, semantic accuracy, or platform superiority. The next research step is a multi-source, multi-node or managed-Kubernetes prototype with delivery, identity, telemetry, incident, and recovery sources; controlled drift, policy denials, incidents, and rollbacks; independent replication; and field evaluation.

References

- [1] Kubernetes, “Controllers,” Kubernetes Documentation, last modified Sept. 1, 2024. [Online]. Available: <https://kubernetes.io/docs/concepts/architecture/controller/>
- [2] Kubernetes, “Auditing,” Kubernetes Documentation, last modified Mar. 16, 2025. [Online]. Available: <https://kubernetes.io/docs/tasks/debug/debug-cluster/audit/>
- [3] Joint Task Force, Security and Privacy Controls for Information Systems and Organizations, NIST SP 800-53 Rev. 5, Release 5.2.0, 2025. doi: 10.6028/NIST.SP.800-53r5. <https://doi.org/10.6028/NIST.SP.800-53r5>
- [4] National Institute of Standards and Technology, The NIST Cybersecurity Framework (CSF) 2.0, Feb. 2024. doi: 10.6028/NIST.CSWP.29. <https://doi.org/10.6028/NIST.CSWP.29>
- [5] Cloud Native Computing Foundation, “CloudEvents—Version 1.0.2,” vendor-neutral event-data specification. [Online]. Available: <https://github.com/cloudevents/spec/blob/v1.0.2/cloudevents/spec.md>
- [6] OpenTelemetry, “Logs Data Model,” OpenTelemetry Specification, status: Stable. Accessed Aug. 5, 2026. [Online]. Available: <https://opentelemetry.io/docs/specs/otel/logs/data-model/>
- [7] L. Moreau and P. Missier, eds., PROV-DM: The PROV Data Model, W3C Recommendation, Apr. 30, 2013. [Online]. Available: <https://www.w3.org/TR/prov-dm/>
- [8] T. Lebo, S. Sahoo, and D. McGuinness, eds., PROV-O: The PROV Ontology, W3C Recommendation, Apr. 30, 2013. [Online]. Available: <https://www.w3.org/TR/prov-o/>
- [9] B. H. Sigelman et al., “Dapper, a Large-Scale Distributed Systems Tracing Infrastructure,” Google Research, technical report, 2010. [Online]. Available: <https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/>
- [10] J. Mace, R. Roelke, and R. Fonseca, “Pivot Tracing: Dynamic Causal Monitoring for Distributed Systems,” in Proc. 25th ACM Symp. Operating Systems Principles, 2015, pp. 378–393. doi: 10.1145/2815400.2815415. <https://doi.org/10.1145/2815400.2815415>
- [11] K.-K. Muniswamy-Reddy, D. A. Holland, U. Braun, and M. Seltzer, “Provenance-Aware Storage Systems,” in Proc. USENIX Annual Technical Conference, 2006. [Online]. Available: <https://www.usenix.org/conference/2006-usenix-annual-technical-conference/provenance-aware-storage-systems>
- [12] A. Bates, D. (Jing) Tian, K. R. B. Butler, and T. Moyer, “Trustworthy Whole-System Provenance for the Linux Kernel,” in Proc. 24th USENIX Security Symposium, 2015, pp. 319–334. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/bates>
- [13] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kubernetes,” Communications of the ACM, vol. 59, no. 5, pp. 50–57, 2016. doi: 10.1145/2890784. <https://doi.org/10.1145/2890784>
- [14] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design Science in Information Systems Research,” MIS Quarterly, vol. 28, no. 1, pp. 75–105, 2004. doi: 10.2307/25148625. <https://doi.org/10.2307/25148625>
- [15] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A Design Science Research Methodology for Information Systems Research,” Journal of Management Information Systems, vol. 24, no. 3, pp. 45–77, 2007. doi: 10.2753/MIS0742-1222240302. <https://doi.org/10.2753/MIS0742-1222240302>
- [16] R. Ross, V. Pillitteri, R. Graubart, D. Bodeau, and R. McQuaid, Developing Cyber-Resilient Systems: A Systems Security Engineering Approach, NIST SP 800-160 Vol. 2 Rev. 1, Dec. 2021. doi: 10.6028/NIST.SP.800-160v2r1. <https://doi.org/10.6028/NIST.SP.800-160v2r1>
- [17] National Institute of Standards and Technology, Secure Hash Standard (SHS), FIPS PUB 180-4, Aug. 2015. doi: 10.6028/NIST.FIPS.180-4. <https://doi.org/10.6028/NIST.FIPS.180-4>
- [18] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, eds., Site Reliability Engineering: How Google Runs Production Systems. Sebastopol, CA: O’Reilly Media, 2016. [Online]. Available: <https://sre.google/sre-book/table-of-contents/>
- [19] O. B. Rocha da Silva, “Resiliência Digital como Vantagem Institucional: Um Modelo de Governança para Maturidade Cloud e DevOps em Instituições Públicas, Educacionais e Comunitárias,” Revista CIA—Publicações Científicas (Ciclo do Conhecimento), July 21, 2026. [Online]. Available: [Article landing page](#)

- [20] OpenTelemetry, “Collector,” OpenTelemetry Documentation. Accessed Aug. 5, 2026. [Online]. Available: <https://opentelemetry.io/docs/collector/>
- [21] OpenTelemetry Collector Contrib, “File Log Receiver,” version 0.158.0. [Online]. Available: <https://github.com/open-telemetry/opentelemetry-collector-contrib/blob/v0.158.0/receiver/filelogreceiver/README.md>
- [22] OpenTelemetry Collector Contrib, “File Exporter,” version 0.158.0. [Online]. Available: <https://github.com/open-telemetry/opentelemetry-collector-contrib/blob/v0.158.0/exporter/fileexporter/README.md>
- [23] Kubernetes, “Audit Configuration API (audit.k8s.io/v1),” Kubernetes Documentation. Accessed Aug. 5, 2026. [Online]. Available: <https://kubernetes.io/docs/reference/config-api/apiserver-audit.v1/>
- [24] Kubernetes SIGs, “Auditing with kind,” kind Documentation. Accessed Aug. 5, 2026. [Online]. Available: <https://kind.sigs.k8s.io/docs/user/auditing/>
- [25] OpenTelemetry, “OpenTelemetry Collector Contrib 0.158.0,” release notes, Aug. 4, 2026. [Online]. Available: <https://github.com/open-telemetry/opentelemetry-collector-releases/releases/tag/v0.158.0>